

IIT GANDHINAGAR

CS202 – Software Tools and Techniques for CSE

Assignment 3

Nishit Mistry (22110172)

Rutvi Shah (22110227)

Course Instructor: Prof. Shouvick Mondal
Date: Saturday 8th November, 2025

Contents

Laboratory Session 9: Development of C# Console Applications	2
Introduction	2
Tools and Technologies	2
Setup and Environment	2
How to Build and Run	2
Activities	3
Activity 1: Hello World	3
Activity 2: OOP Calculator	3
Activity 3: Loops and Functions	5
Activity 4: Arrays	9
Output Reasoning	13
Level 0	13
Level 1	14
Level 2	15
Results	16
Conclusions	16
References	16
 Lab Assignment 10: Object Lifecycle, Inheritance, and Event-Driven Program-	
ming in C#	17
0.1 Objective and Learning Outcomes	17
0.2 Lab Implementation and Analysis	17
0.3 Discussion:	29

Laboratory Session 9

Introduction

This laboratory focuses on building C# console applications with the .NET SDK and Visual Studio. The aim is to practice basic syntax, control structures, functions, and object-oriented programming (OOP) principles by implementing four activities and reasoning about program outputs.

Objectives

- Set up and use Visual Studio / .NET CLI
- Build and run C# console apps
- Implement loops, conditionals, and functions
- Apply OOP concepts in simple programs
- Practice arrays, sorting, and matrix operations
- Analyze code behavior and explain outputs

Tools and Technologies

- SDK: .NET 8 (LTS) (project targets updated to `net8.0`)
- Language: C# (latest stable)
- CLI: `dotnet` (build/run), optional `csc` for direct compilation

Setup and Environment

How to Build and Run

Open a Command Prompt and run:

```
1 cd ".\Activity1_HelloWorld"  
2 dotnet run
```

Similarly for other activities. For Activity 3 non-interactive run (example):

```
1 (echo exit & echo 5) | dotnet run --project ".\  
    Activity3_LoopsAndFunctions.csproj"
```

Activities

Activity 1: Hello World

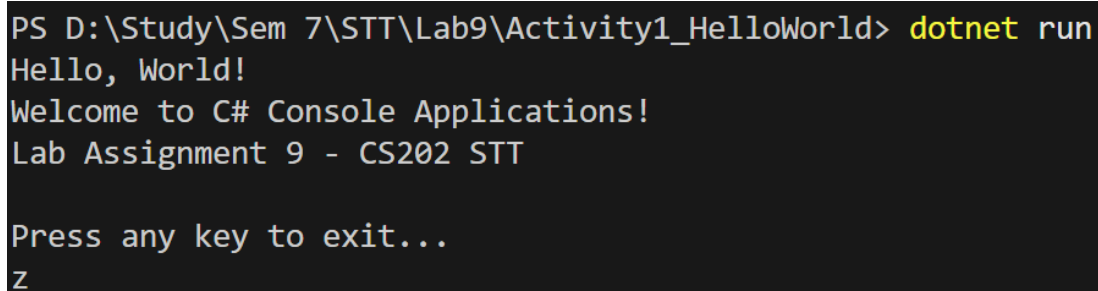
Goal: Basic console app output.

Key Concepts: Program structure, entry point, console I/O.

Listing 1: Program.cs (excerpt)

```
1 Console.WriteLine("Hello, World!");
2 Console.WriteLine("Welcome to C# Console Applications!");
3 Console.WriteLine("Lab Assignment 9 - CS202 STT");
```

Output:



```
PS D:\Study\Sem 7\STT\Lab9\Activity1_HelloWorld> dotnet run
Hello, World!
Welcome to C# Console Applications!
Lab Assignment 9 - CS202 STT

Press any key to exit...
Z
```

Activity 2: OOP Calculator

Goal: Object-oriented program that accepts two numbers, performs arithmetic, and checks if the sum is even/odd.

Key Concepts: Classes, encapsulation, methods, conditionals.

Listing 2: Calculator class (excerpt)

```
1 class Calculator
2 {
3     private double num1;
4     private double num2;
5     // Constructor
6     public Calculator(double number1, double number2)
7     {
8         num1 = number1;
9         num2 = number2;
10    }
11    // Method for addition
12    public double Add()
13    {
14        return num1 + num2;
15    }
16    // Method for subtraction
17    public double Subtract()
18    {
19        return num1 - num2;
20    }
```

```
21 // Method for multiplication
22 public double Multiply()
23 {
24     return num1 * num2;
25 }
26 // Method for division
27 public double Divide()
28 {
29     if (num2 == 0)
30     {
31         Console.WriteLine("Error: Division by zero!");
32         return 0;
33     }
34     return num1 / num2;
35 }
36 // Method to check if sum is even or odd
37 public void CheckSumEvenOdd()
38 {
39     double sum = Add();
40
41     if (sum % 2 == 0)
42     {
43         Console.WriteLine($"The sum {sum} is EVEN.");
44     }
45     else
46     {
47         Console.WriteLine($"The sum {sum} is ODD.");
48     }
49 }
50 // Method to display all results
51 public void DisplayResults()
52 {
53     Console.WriteLine("\n==== Calculation Results =====");
54     Console.WriteLine($"Number 1: {num1}");
55     Console.WriteLine($"Number 2: {num2}");
56     Console.WriteLine($"Addition: {num1} + {num2} = {Add()}");
57     Console.WriteLine($"Subtraction: {num1} - {num2} = {Subtract()}");
58     Console.WriteLine($"Multiplication: {num1} * {num2} = {Multiply()}");
59     Console.WriteLine($"Division: {num1} / {num2} = {Divide()}");
60     CheckSumEvenOdd();
61     Console.WriteLine("=====\n");
62 }
63 }
64
65 class Program
66 {
67     static void Main(string[] args)
68     {
69         Console.WriteLine("==== Object-Oriented Calculator =====\n");
```

```

70      // Accept user input for two numbers
71      Console.WriteLine("Enter the first number: ");
72      double number1 = Convert.ToDouble(Console.ReadLine());
73      Console.WriteLine("Enter the second number: ");
74      double number2 = Convert.ToDouble(Console.ReadLine());
75      // Create Calculator object
76      Calculator calc = new Calculator(number1, number2);
77      // Display all results
78      calc.DisplayResults();
79      Console.WriteLine("Press any key to exit...");
80      Console.ReadKey();
81  }
82 }

```

Output:

```

PS D:\Study\Sem 7\STT\Lab9\Activity2_Calculator> dotnet run
===== Object-Oriented Calculator =====

Enter the first number: 5
Enter the second number: 7

===== Calculation Results =====
Number 1: 5
Number 2: 7
Addition: 5 + 7 = 12
Subtraction: 5 - 7 = -2
Multiplication: 5 * 7 = 35
Division: 5 / 7 = 0.7142857142857143
The sum 12 is EVEN.
=====

Press any key to exit...
Z

```

Activity 3: Loops and Functions

Goal: Demonstrate for, foreach, do-while loops and implement factorial as a function.

Refactor Highlights: Robust input handling, EOF safety, overflow-safe factorial, non-interactive friendly pause.

Listing 3: Robust factorial and input prompt (excerpt)

```

1  class LoopOperations
2  {
3      // Method to demonstrate for loop
4      public void ForLoopDemo()
5      {
6          Console.WriteLine("\n===== For Loop (1 to 10) =====");
7          for (int i = 1; i <= 10; i++)
8          {

```

```
9         Console.Write(i + " ");
10     }
11     Console.WriteLine();           Console.WriteLine("
=====\\n");
12 }
13 // Method to demonstrate foreach loop
14 public void ForEachLoopDemo()
15 {
16     Console.WriteLine("==== Foreach Loop (1 to 10) =====");
17     int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
18     foreach (int num in numbers)
19     {
20         Console.Write(num + " ");
21     }
22     Console.WriteLine();           Console.WriteLine("
=====\\n");
23 }
24 // Method to demonstrate do-while loop
25 public void DoWhileLoopDemo()
26 {
27     Console.WriteLine("==== Do-While Loop =====");
28     Console.WriteLine("Enter text (type 'exit' to quit):");
29     string input;
30     do
31     {
32         Console.Write("> ");
33         input = Console.ReadLine();
34         // Treat Ctrl+Z (EOF) as an exit
35         if (input is null)
36         {
37             Console.WriteLine("(No input detected - exiting loop.)")
38             ;
39             break;
40         }
41         input = input.Trim();
42         if (!input.Equals("exit", StringComparison.OrdinalIgnoreCase))
43         {
44             if (input.Length == 0)
45             {
46                 Console.WriteLine("(Empty input - type text or 'exit
' to quit.)");
47             }
48             else
49             {
50                 Console.WriteLine($"You entered: {input}");
51             }
52         } while (!string.Equals(input, "exit", StringComparison.
OrdinalIgnoreCase));
53     Console.WriteLine("Exiting do-while loop...");
```

```
54         Console.WriteLine("=====\n");
55     }
56     // Static method to calculate factorial (safe with overflow check)
57     public static long CalculateFactorial(int n)
58     {
59         if (n < 0)
60         {
61             Console.WriteLine("Factorial is not defined for negative
62                 numbers.");
63             return -1;
64         }
65         long factorial = 1;
66         try
67         {
68             checked
69             {
70                 for (int i = 2; i <= n; i++)
71                 {
72                     factorial *= i;
73                 }
74             }
75         } catch (OverflowException)
76         {
77             Console.WriteLine("Overflow: The factorial is too large to
78                 fit in a 64-bit integer. Try a smaller n (<= 20).");
79             return -1;
80         }
81         return factorial;
82     }
83
84     class Program
85     {
86         static void Main(string[] args)
87         {
88             Console.WriteLine("==== Loops and Functions Demo =====\n");
89             // Create object for loop demonstrations
90             LoopOperations loopOps = new LoopOperations();
91             // Demonstrate for loop
92             loopOps.ForLoopDemo();
93             // Demonstrate foreach loop
94             loopOps.ForEachLoopDemo();
95             // Demonstrate do-while loop
96             loopOps.DoWhileLoopDemo();
97             // Factorial calculation with robust input parsing
98             Console.WriteLine("==== Factorial Calculation =====");
99             int number = PromptForInt("Enter a non-negative integer (
100                 recommended 0..20) to calculate its factorial: ", 0);
101             long result = LoopOperations.CalculateFactorial(number);
102             if (result != -1)
```



```
102     {
103         Console.WriteLine($"Factorial of {number} = {result}");
104     } Console.WriteLine("=====\n");
105     ;
106     PauseIfInteractive();
107 }
108 private static int PromptForInt(string prompt, int minInclusive)
109 {
110     while (true)
111     {
112         Console.Write(prompt);
113         var line = Console.ReadLine();
114         if (line is null)
115         {
116             Console.WriteLine("(No input detected - defaulting to 0.)");
117             return 0;
118         }
119         line = line.Trim();
120         if (int.TryParse(line, out int value))
121         {
122             if (value < minInclusive)
123             {
124                 Console.WriteLine($"Please enter an integer >= {minInclusive}.");
125                 continue;
126             }
127             return value;
128         }
129         Console.WriteLine("Invalid number. Please enter a valid integer.");
130     }
131 }
132 private static void PauseIfInteractive()
133 {
134     try
135     {
136         if (!Console.IsInputRedirected)
137         {
138             Console.WriteLine("Press any key to exit...");
139             Console.ReadKey(true);
140         }
141     }
142     catch
143     {
144         // Non-interactive environment; skip pause
145     }
146 }
```

Output:

```

PS D:\Study\Sem 7\STT\Lab9\Activity3_LoopsAndFunctions> dotnet run
===== Loops and Functions Demo =====

===== For Loop (1 to 10) =====
1 2 3 4 5 6 7 8 9 10
=====

===== Foreach Loop (1 to 10) =====
1 2 3 4 5 6 7 8 9 10
=====

===== Do-While Loop =====
Enter text (type 'exit' to quit):
> s
You entered: s
> ac
You entered: ac
> afe
You entered: afe
> exit
Exiting do-while loop...
=====

===== Factorial Calculation =====
Enter a non-negative integer (recommended 0..20) to calculate its factorial: 5
Factorial of 5 = 120
=====

Press any key to exit...

```

Activity 4: Arrays

Goal: Implement bubble sort (no library calls), 2D $\rightarrow$ 1D conversions (row/column-major), and matrix multiplication.

Listing 4: Bubble sort (excerpt)

```

1 class ArrayOperations
2 {
3     // Method to implement Bubble Sort
4     public void BubbleSort(int[] arr)
5     {
6         int n = arr.Length;
7         Console.WriteLine("\nArray before sorting:");
8         PrintArray(arr);
9         // Bubble Sort Algorithm
10        for (int i = 0; i < n - 1; i++)
11        {
12            for (int j = 0; j < n - i - 1; j++)
13            {
14                if (arr[j] > arr[j + 1])
15                {
16                    // Swap arr[j] and arr[j+1]

```

```
17         int temp = arr[j];
18         arr[j] = arr[j + 1];
19         arr[j + 1] = temp;
20     }
21 }
22 }
23 Console.WriteLine("Array after sorting:");
24 PrintArray(arr);
25 }
26 // Method to print 1D array
27 public void PrintArray(int[] arr)
28 {
29     foreach (int element in arr)
30     {
31         Console.Write(element + " ");
32     }
33     Console.WriteLine();
34 }
35 // Method to convert 2D array to 1D array in Row Major Order
36 public int[] ConvertToRowMajor(int[,] arr2D)
37 {
38     int rows = arr2D.GetLength(0);
39     int cols = arr2D.GetLength(1);
40     int[] arr1D = new int[rows * cols];
41     int index = 0;
42     for (int i = 0; i < rows; i++)
43     {
44         for (int j = 0; j < cols; j++)
45         {
46             arr1D[index++] = arr2D[i, j];
47         }
48     }
49     return arr1D;
50 }
51 // Method to convert 2D array to 1D array in Column Major Order
52 public int[] ConvertToColumnMajor(int[,] arr2D)
53 {
54     int rows = arr2D.GetLength(0);
55     int cols = arr2D.GetLength(1);
56     int[] arr1D = new int[rows * cols];
57     int index = 0;
58     for (int j = 0; j < cols; j++)
59     {
60         for (int i = 0; i < rows; i++)
61         {
62             arr1D[index++] = arr2D[i, j];
63         }
64     }
65     return arr1D;
66 }
67 // Method to print 2D array
```

```
68     public void Print2DArray(int[,] arr)
69     {
70         int rows = arr.GetLength(0);
71         int cols = arr.GetLength(1);
72
73         for (int i = 0; i < rows; i++)
74         {
75             for (int j = 0; j < cols; j++)
76             {
77                 Console.Write(arr[i, j] + "\t");
78             }
79             Console.WriteLine();
80         }
81     }
82     // Method to multiply two matrices
83     public int[,] MatrixMultiplication(int[,] A, int[,] B)
84     {
85         int rowsA = A.GetLength(0);
86         int colsA = A.GetLength(1);
87         int rowsB = B.GetLength(0);
88         int colsB = B.GetLength(1);
89         // Check if multiplication is possible
90         if (colsA != rowsB)
91         {
92             Console.WriteLine("Matrix multiplication not possible!");
93             Console.WriteLine($"Columns of A ({colsA}) must equal rows
94                 of B ({rowsB})");
95             return null;
96         }
97         // Result matrix C will be of size rowsA x colsB
98         int[,] C = new int[rowsA, colsB];
99         // Perform multiplication
100        for (int i = 0; i < rowsA; i++)
101        {
102            for (int j = 0; j < colsB; j++)
103            {
104                C[i, j] = 0;
105                for (int k = 0; k < colsA; k++)
106                {
107                    C[i, j] += A[i, k] * B[k, j];
108                }
109            }
110        }
111        return C;
112    }
113
114    class Program
115    {
116        static void Main(string[] args)
117        {
```

```
118 Console.WriteLine("==== Array Operations Demo ====\\n");
119 ArrayOperations arrayOps = new ArrayOperations();
120 // ===== Bubble Sort =====
121 Console.WriteLine("===== 1. Bubble Sort =====");
122 int[] arr = { 64, 34, 25, 12, 22, 11, 90 };
123 arrayOps.BubbleSort(arr);
124 Console.WriteLine();
125 // ===== 2D to 1D Array Conversion =====
126 Console.WriteLine("===== 2. 2D to 1D Array Conversion =====");
127 int[,] arr2D = {
128     { 1, 2, 3 },
129     { 4, 5, 6 },
130     { 7, 8, 9 }
131 };
132 Console.WriteLine("\\nOriginal 2D Array:");
133 arrayOps.Print2DArray(arr2D);
134 Console.WriteLine("\\n(i) Row Major Order:");
135 int[] rowMajor = arrayOps.ConvertToRowMajor(arr2D);
136 arrayOps.PrintArray(rowMajor);
137 Console.WriteLine("\\n(ii) Column Major Order:");
138 int[] colMajor = arrayOps.ConvertToColumnMajor(arr2D);
139 arrayOps.PrintArray(colMajor);
140 Console.WriteLine();
141 // ===== Matrix Multiplication =====
142 Console.WriteLine("===== 3. Matrix Multiplication =====");
143 int[,] matrixA = {
144     { 1, 2, 3 },
145     { 4, 5, 6 }
146 };
147 int[,] matrixB = {
148     { 7, 8 },
149     { 9, 10 },
150     { 11, 12 }
151 };
152 Console.WriteLine("\\nMatrix A:");
153 arrayOps.Print2DArray(matrixA);
154 Console.WriteLine("\\nMatrix B:");
155 arrayOps.Print2DArray(matrixB);
156 int[,] matrixC = arrayOps.MatrixMultiplication(matrixA, matrixB)
157 ;
158 if (matrixC != null)
159 {
160     Console.WriteLine("\\nMatrix C (A B):");
161     arrayOps.Print2DArray(matrixC);
162 }
163 Console.WriteLine("\\nPress any key to exit...");
164 Console.ReadKey();
165 }
```

Output:

```
PS D:\Study\Sem 7\STT\Lab9\Activity4_Arrays> dotnet run
===== Array Operations Demo =====

===== 1. Bubble Sort =====

Array before sorting:
64 34 25 12 22 11 90
Array after sorting:
11 12 22 25 34 64 90

===== 2. 2D to 1D Array Conversion =====

Original 2D Array:
1      2      3
4      5      6
7      8      9

(i) Row Major Order:
1 2 3 4 5 6 7 8 9

(ii) Column Major Order:
1 4 7 2 5 8 3 6 9

===== 3. Matrix Multiplication =====

Matrix A:
1      2      3
4      5      6

Matrix B:
7      8
9      10
11     12

Matrix C (A × B):
58     64
139    154

Press any key to exit...
```

Output Reasoning

Level 0

Listing 5: Level 0 Example 1

```
1 using System;
2 class Program
3 {
```

```
4     public static void Main(string[] args)
5     {
6         int a = 0;
7         Console.WriteLine(a++);
8     }
9 }
```

Reason: Post-increment prints current value then increments (output: 0)

Listing 6: Level 0 Example 2

```
1 using System;
2 class Program
3 {
4     public void Main(string[] args)
5     {
6         int a = 0;
7         Console.WriteLine(a++);
8     }
9 }
```

Reason: Non-static Main cannot be an entry point (compile/run error)

Level 1

Listing 7: Level 1 Example 1

```
1 class Program
2 {
3     public static void Main(string[] args)
4     {
5         int a = 0;
6         int b = a++;
7         Console.WriteLine(a++.ToString(), ++a, -a++);
8         Console.WriteLine((a++).ToString() + (-a++).ToString());
9         Console.WriteLine(~b);
10    }
11 }
```

Reason: Pre/post increments and formatting: multi-argument WriteLine with a plain string uses only the first argument

Listing 8: Level 1 Example 2

```
1 using System;
2 /*you can also write top level code outside of a class. C# takes
3 care of this by providing internal entry point Main*/
4 Console.WriteLine("int x = 3;");
5 Console.WriteLine("int y = 2 + ++x;");
6 int x = 3; //default visibility is internal (outside a class)
7 int y = 2 + ++x;
8 Console.WriteLine($"x = {x} and y = {y}");
9 Console.WriteLine("x = 3 << 2;");
```

```
10 Console.WriteLine("y = 10 >> 1;");
11 x = 3 << 2;
12 y = 10 >> 1;
13 Console.WriteLine($"x = {x} and y = {y}");
14 x = ~x;
15 y = ~y;
16 Console.WriteLine($"x = {x} and y = {y}");
```

Reason: Bitwise: shifts and bitwise NOT: $\sim n = -(n + 1)$

Level 2

Listing 9: Level 2 Example 1

```
1 using System;
2 public class Program
3 {
4     static void Main()
5     {
6         try
7         {
8             int i=int.MaxValue;
9             Console.WriteLine(-(i+1)-i);
10            for(i=0; i<=int.MaxValue;i++); //note semicolon here
11            Console.WriteLine("Program ended!");
12        }
13        catch(Exception ex)
14        {
15            Console.WriteLine(ex.ToString());
16        }
17    }
18 }
```

Reason: Overflow wraps in default unchecked arithmetic; a for-loop with overflow on the counter can become effectively infinite

Listing 10: Level 2 Example 2

```
1 using System;
2 public class Program
3 {
4     static void Main(string[] args)
5     {
6         Main(["CS202"]);
7     }
8 }
```

Reason: Unbounded recursion without base case causes a StackOverflowException

Results

All four activities compile and run targeting .NET 8. Activity 3 was refactored for correctness and robust user interaction. Array algorithms produce correct results (bubble sort ordering, 2D $\rightarrow$ 1D conversions, and valid matrix multiplication outputs). Output reasoning examples are accompanied by explanations and expected outputs.

Conclusions

The lab meets the learning outcomes: environment setup, basic C# syntax, loops and conditionals, function design, OOP usage, and array algorithm implementations. The refactor of Activity 3 improved resilience to invalid input and non-interactive execution.

References

- Microsoft .NET and C# Docs: <https://learn.microsoft.com/dotnet/>
- C# Language Reference: <https://learn.microsoft.com/dotnet/csharp/language-reference/>
- Github repo link https://github.com/kneeshit/STT_Assignments/tree/main/Lab9

Laboratory Session 10

0.1 Objective and Learning Outcomes

The primary objective of this lab was to reinforce object-oriented principles by implementing encapsulation, inheritance, and polymorphism in C# console applications. A secondary focus involved understanding object management through constructors and destructors, and observing dynamic method dispatch in an inheritance hierarchy.

Learning Outcomes:

- Created and managed class instances, including explicit tracking of object lifecycle using a static counter.
- Designed a class hierarchy (**Vehicle**, **Car**, **Truck**) demonstrating inheritance and method overriding.
- Analyzed C# code snippets involving operator precedence, bitwise operations, delegates, events, and reference semantics.

0.2 Lab Implementation and Analysis

The lab consisted of two main tasks: Object Lifecycle Management and Object-Oriented Polymorphism.

Part 1: Constructors and Data Control

To implement a class **Program** that demonstrates constructors, destructors, and data handling with a static counter to track active objects.

Code Implementation

Listing 11: Program class demonstrating constructors, destructors, and data control

```
1 using System;
2
3 public class Program
4 {
5     private int data;
6     private static int counter = 0;
7
8     // Constructor
```

```
9      public Program()  
10     {  
11         counter++;  
12         Console.WriteLine("Constructor Called. Active objects: " +  
13             counter);  
14     }  
15     // Destructor  
16     ~Program()  
17     {  
18         counter--;  
19         Console.WriteLine("Object Destroyed. Active objects: " + counter  
20             );  
21     }  
22     public void set_data(int x)  
23     {  
24         data = x;  
25     }  
26  
27     public void show_data()  
28     {  
29         Console.WriteLine("Data value: " + data);  
30     }  
31  
32     public static void Main(string[] args)  
33     {  
34         Program p1 = new Program();  
35         Program p2 = new Program();  
36         Program p3 = new Program();  
37  
38         p1.set_data(10);  
39         p2.set_data(20);  
40         p3.set_data(30);  
41  
42         p1.show_data();  
43         p2.show_data();  
44         p3.show_data();  
45  
46         // Objects go out of scope when program ends, triggering  
47             destructor  
48     }
```

Explanation

- Each time a `Program` object is created, the constructor prints “Constructor Called” and increments the static counter.
- When objects are destroyed (automatically at the end of program execution), the destructor

prints “Object Destroyed” and decrements the counter.

- The counter tracks the number of active objects currently existing in memory.

Observation

During program execution:

- Three constructors are called sequentially as objects are created.
- When the program ends, destructors are called in reverse order of creation, showing object destruction.

Output Snapshot

```
Constructor Called. Active objects: 1
Constructor Called. Active objects: 2
Constructor Called. Active objects: 3
Data value: 10
Data value: 20
Data value: 30
Object Destroyed. Active objects: 2
Object Destroyed. Active objects: 1
Object Destroyed. Active objects: 0
```

Part 2: Inheritance and Method Overriding

To demonstrate how method overriding works in inheritance and how runtime polymorphism ensures that the correct derived class methods are called even when accessed via base class references.

Code Implementation

Listing 12: Inheritance and method overriding demonstration

```
1 using System;
2
3 public class Vehicle
4 {
5     protected int speed;
6     protected int fuel;
7
8     public Vehicle(int s, int f)
9     {
10         speed = s;
11         fuel = f;
12     }
13
14     public virtual void Drive()
15     {
```

```
16         fuel -= 5;
17         Console.WriteLine("Vehicle is moving...");
18     }
19
20     public virtual void ShowInfo()
21     {
22         Console.WriteLine("Speed: " + speed + " km/h, Fuel: " + fuel);
23     }
24 }
25
26 public class Car : Vehicle
27 {
28     private int passengers;
29
30     public Car(int s, int f, int p) : base(s, f)
31     {
32         passengers = p;
33     }
34
35     public override void Drive()
36     {
37         fuel -= 10;
38         Console.WriteLine("Car is moving with passengers...");
39     }
40
41     public override void ShowInfo()
42     {
43         Console.WriteLine("Speed: " + speed + " km/h, Fuel: " + fuel + "
44             , Passengers: " + passengers);
45     }
46 }
47
48 public class Truck : Vehicle
49 {
50     private int cargoWeight;
51
52     public Truck(int s, int f, int c) : base(s, f)
53     {
54         cargoWeight = c;
55     }
56
57     public override void Drive()
58     {
59         fuel -= 15;
60         Console.WriteLine("Truck is moving with cargo...");
61     }
62
63     public override void ShowInfo()
64     {
65         Console.WriteLine("Speed: " + speed + " km/h, Fuel: " + fuel + "
66             , Cargo: " + cargoWeight + " kg");
```

```
65     }
66 }
67
68 public class Test
69 {
70     public static void Main(string[] args)
71     {
72         Vehicle[] vehicles = new Vehicle[3];
73         vehicles[0] = new Vehicle(60, 100);
74         vehicles[1] = new Car(80, 120, 4);
75         vehicles[2] = new Truck(50, 150, 500);
76
77         foreach (Vehicle v in vehicles)
78         {
79             v.Drive();
80             v.ShowInfo();
81             Console.WriteLine();
82         }
83     }
84 }
```

Explanation

- The base class `Vehicle` defines virtual methods `Drive()` and `ShowInfo()`.
- The derived classes `Car` and `Truck` override these methods to provide specialized behavior.
- When the array of base class references (`Vehicle[]`) calls these methods, the actual derived class versions are executed — this demonstrates **runtime polymorphism**.

Output Snapshot

Vehicle is moving...

Speed: 60 km/h, Fuel: 95

Car is moving with passengers...

Speed: 80 km/h, Fuel: 110, Passengers: 4

Truck is moving with cargo...

Speed: 50 km/h, Fuel: 135, Cargo: 500 kg

Discussion

- Method overriding allows derived classes to redefine base-class behavior.
- Even when using base-class references, the program determines the correct method to execute at runtime (dynamic dispatch).
- Constructors and destructors demonstrate the object lifecycle, and the static counter provides a clear understanding of how many objects are alive at any point.

Part 3: Output Reasoning and Analysis

Level 0: Operator Precedence and Bitwise Operations

Program 1:

Listing 13: Postfix and Prefix Increment with Unary and Bitwise Operators

```

1 using System;
2 int a = 3;
3 int b = a++;
4 Console.WriteLine($"a is {a++}, b is {-++b}");
5 int c = 3;
6 int d = ++c;
7 Console.WriteLine($"c is {-c--}, d is {~d}");

```

Predicted Output:

```

a is 4, b is -4
c is -4, d is -5

```

Reasoning: Initially, $a = 3$. The statement $b = a++$ assigns $b = 3$ and then increments a to 4. Next, $a++$ prints 4 and increments a to 5, while $-++b$ first increments b to 4 and prints -4 . For the second part, $c = 3$ and $++c$ makes it 4, so $d = 4$. The expression $-c--$ prints -4 and then decrements c to 3. The bitwise NOT operator $\sim$ inverts the bits of 4 (00000100) to -5 (two's complement representation).

Program 2:

Listing 14: Constructor Execution and Bitwise Shift Operator

```

1 using System;
2 class Program
3 {
4     int age;
5     Program() => age=age==0?age+1:age-1;
6     static void Main()
7     {
8         int k = "010%".Replace('0','%').Length;
9         Console.Write("[ " + (k<<++new Program().age).ToString() + " ]");
10        Console.Write("[ " + "010%".Split('1')[1][0] + " ]");
11        Console.Write("[ " + "010%".Split('0')[1][0] + " ]");
12        Console.Write("[ " + int.Parse(Convert.ToString("123".ToCharArray()[~-1]))
13            + " ]");
14    }

```

Predicted Output:

```
[16] [0] [1] [1]%
```

Reasoning: $"010%".Replace('0','%')$ gives $%%1%$ with length 4. Thus, $k = 4$. The constructor sets $age = 1$, so $++age = 2$. Therefore, $k \ll 2 = 4 * 2^2 = 16$. $"010%".Split('1')[1][0]$ gives $'0'$, $"010%".Split('0')[1][0]$ gives $'1'$. $"123".ToCharArray()[~-1]$ accesses the last character ($'3'$) since $~-1 = 0$. $int.Parse(Convert.ToString('3'))$ gives 3.

Program 3:

Listing 15: Array Manipulation and In-Place Reordering

```
1 using System;
2 class Program
3 {
4     static void Main()
5     {
6         int[] nums = {0, 1, 0, 3, 12};
7         int pos = 0;
8         for (int i = 0; i < nums.Length; i++)
9         {
10            if (nums[i] != 0)
11            {
12                int temp = nums[pos];
13                nums[pos] = nums[i];
14                nums[i] = temp;
15                pos++;
16            }
17        }
18        Console.WriteLine(string.Join(", ", nums));
19    }
20 }
```

Predicted Output:

1, 3, 12, 0, 0

Reasoning: The loop moves non-zero elements to the front while maintaining their order. After the swaps: Initial $\rightarrow$ [0, 1, 0, 3, 12] Step 1 $\rightarrow$ [1, 0, 0, 3, 12] Step 2 $\rightarrow$ [1, 3, 0, 0, 12] Step 3 $\rightarrow$ [1, 3, 12, 0, 0]

Level 1: Constructor Overloading and Loops

Listing 16: Constructor Overloading and Incremental Loop Control

```
1 using System;
2
3 class Program
4 {
5     int age;
6     Program() => age = age == 0 ? age + 1 : age - 1;
7     static void Main()
8     {
9         int k = "010%".Replace('0', '%').Length;
10        Console.Write "[" + (k << ++new Program().age).ToString() + "]"
11        ;
12        Console.Write "[" + "010%".Split('1')[1][0] + "]";
13        Console.Write "[" + "010%".Split('0')[1][0] + "]";
14        Console.Write "[" + int.Parse(Convert.ToString("123".ToCharArray
15        (~-1))) + "]" );
16    }
```



```
14     }
15 }
```

Predicted Output:

[16] [0] [1] [1]%

Reasoning: Same as before — demonstrates string operations, prefix increment, and left shift precedence.

Program 2:

Listing 17: Nested Loops, Constructors, and Variable Lifetime

```
1 using System;
2 class Program
3 {
4     int f;
5     public static void Main(string[] args)
6     {
7         Console.WriteLine("run 1");
8         Program p = new Program(new int() + "0".Length);
9         for (int i = 0, _ = i; i < 5 && ++p.f >= 0; i++, Console.
10             WriteLine(p.f++)) ;
11         {
12             for (; p.f == 0;) ;
13             {
14                 Console.WriteLine(p.f);
15             }
16             Console.WriteLine("\nrun 2");
17             p = new Program(p.f);
18             Console.WriteLine(p.f);
19             Console.WriteLine("\nrun 3");
20             p = new Program();
21             Console.WriteLine(p.f);
22         }
23         Program() => f = 0;
24         Program(int x) => f = x;
25 }
```

Predicted Output:

run 1

2

4

6

8

10

11

run 2

11

run 3

0

Reasoning: - `new int()` initializes 0; `"0".Length = 1` `f = 1`. - The loop increments `f` and prints its value five times. - Nested blocks print the final `f = 11`. - Subsequent reinitializations show constructor behavior and variable persistence.

Program 3:

Listing 18: Recursion, Delegates, and Null-Conditional Operator

```
1 public class A
2 {
3     public virtual void f1() { Console.WriteLine("f1"); }
4 }
5 public class B:A
6 {
7     public override void f1() => Console.WriteLine("f2");
8 }
9 class Program
10 {
11     static int i=0;
12     public event funcPtr handler;
13     public delegate void funcPtr();
14     public void destroy()
15     {
16         if (i == 6) return;
17         else
18         {
19             Console.WriteLine(i++);
20             destroy();
21         }
22     }
23     public static void Main(string[] args)
24     {
25         Program p = new Program();
26         p.handler += new funcPtr((new A()).f1);
27         p.handler += new funcPtr((new B()).f1);
28         p.handler();
29         p.handler -= new funcPtr((new B()).f1);
30         p.handler -= new funcPtr((new A()).f1);
31         p?.destroy();
32         p = null;
33         i = -6;
34         p?.destroy();
35         (new Program()).?.destroy();
36     }
37 }
```

Predicted Output:

f1
f2
0
1
2
3
4
5
-6
-5
-4
-3
-2
-1
0
1
2
3
4
5

Reasoning: The delegate initially calls both `f1()` and `f2()`. `destroy()` recursively prints values of `i` until it reaches 6. The null-conditional operator (`?.`) safely checks if `p` is not null before calling `destroy()`, demonstrating short-circuit behavior. When `i = -6`, recursion runs until `i = 6` again.

Level 2: Inheritance and Delegates

Program 1:

Listing 19: Inheritance and Method Overriding Behavior

```
1 public class Institute
2 {
3     internal int i = 7;
4     public Institute() { Console.Write("1"); }
5     public virtual void info() { Console.Write("2"); }
6 }
7 public class IITGN:Institute
8 {
9     public int i = 8;
10    public IITGN() { Console.Write("3"); }
11    public IITGN(int i) { Console.Write("4"); }
12    public override void info() { Console.Write("5"); }
13 }
14 class Program
15 {
16     public static void Main(string[] args)
17     {
```

```
18 Console.Write("6");
19 Institute ins1 = new Institute();
20 ins1.info();
21 IITGN ab101 = new IITGN(3);
22 ab101 = new IITGN();
23 ab101.info();
24 Console.WriteLine();
25 Console.WriteLine(ab101.i);
26 Console.WriteLine(~(((Institute)ab101).i));
27 }
28 }
```

Predicted Output:

61214135

8

-8

Reasoning: 1. Prints 6 first. 2. Creating new Institute() → constructor prints 1, then info() prints 2. 3. Creating new IITGN(3) → parent constructor (1) + child constructor (4). 4. Creating new IITGN() → parent 1 + child 3. 5. info() prints overridden 5. 6. ab101.i = 8, while ((Institute)ab101).i = 7. 7 = -8 (bitwise inversion).

Program 2:

Listing 20: Delegate Invocation and Combination

```
1 using System;
2 public class Program
3 {
4     public delegate void mydel();
5     public void fun1() { Console.WriteLine("fun1()"); }
6     public void fun2() { Console.WriteLine("fun2()"); }
7     public static void Main(string[] args)
8     {
9         Program p = new Program();
10        mydel obj1 = new mydel(p.fun1);
11        obj1 += new mydel(p.fun2);
12        Console.WriteLine("run 1");
13        obj1();
14        mydel obj2 = new mydel(p.fun2);
15        obj2 += new mydel(p.fun1);
16        Console.WriteLine("run 2");
17        obj2();
18        obj2 -= p.fun2;
19        Console.WriteLine("run 3");
20        obj2();
21    }
22 }
```

Predicted Output:

run 1

```
fun1()
fun2()
run 2
fun2()
fun1()
run 3
fun1()
```

Reasoning: Delegates can hold multiple method references. When combined using `+=`, they form a chain that executes sequentially. Removing a delegate reference (`-=`) excludes it from future invocations.

Program 3:

Listing 21: Object References and ArrayList Manipulation

```
1 using System;
2 using System.Collections;
3
4 public class Program
5 {
6     int x;
7
8     public static void Main(string[] args)
9     {
10         ArrayList L = new ArrayList();
11         L.Add(new Program());
12         L.Add(new Program());
13
14         // First loop
15         for (int i = 0; i < L.Count; i++)
16             Console.WriteLine(++((Program)L[i]).x);
17
18         // Replace first element with second
19         L[0] = L[1];
20         ((Program)L[0]).x = 202;
21
22         // Second loop
23         for (int i = 0; i < L.Count; i++)
24             Console.WriteLine(((Program)L[i]).x);
25
26         // Modify first element and remove it
27         ((Program)L[0]).x = 111;
28         L.RemoveAt(0);
29
30         Console.WriteLine(L.Count);
31         Console.WriteLine(((Program)L[0]).x);
32     }
33 }
```

Predicted Output:

1

```
1
202
202
1
111
```

Reasoning: Two `Program` objects are created, both with `x = 0`. The first loop increments and prints both as 1. Replacing `L[0]` with `L[1]` means both refer to the same object. When `x = 202` is set, both elements show 202. After modifying `x = 111` and removing the first element, the remaining element retains that updated value.

0.3 Discussion:

This lab provided a practical understanding of object-oriented principles in `C#`, focusing on how constructors, destructors, inheritance, and events operate at runtime. Through the first task, students observed how constructors and destructors control object lifecycle and memory management in managed environments like `.NET`. The inclusion of a static counter demonstrated how shared data across class instances can track active objects, reinforcing encapsulation and class-level scope concepts.

In the inheritance section, polymorphism was observed through the overriding of virtual methods `Drive()` and `ShowInfo()` in derived classes `Car` and `Truck`. Despite using base-class references, the derived class implementations were executed, demonstrating runtime polymorphism and dynamic binding.

The event-driven programming exercises illustrated the publisher-subscriber model using delegates and event handlers. Students learned how event invocation order depends on the registration sequence and how removal of event handlers changes runtime behavior. These exercises reinforced how events can decouple components and enable reactive systems.

In the reasoning problems, analysis of increment operators, bitwise operations, and string manipulations in `C#` improved understanding of operator precedence and evaluation order. Tracing outputs helped identify subtle differences between pre/post increment, logical operators, and expression parsing in `C#` syntax.

Conclusion

This lab deepened the understanding of core object-oriented concepts in `C#`. It demonstrated how constructors, destructors, inheritance, and events work together to create structured and reusable code. Students observed object lifecycle behavior, runtime polymorphism, and event handling in action. Overall, the lab effectively linked theory with practical implementation, strengthening both conceptual and coding skills.

References

- Lecture 10 Slides
- Lecture 11 Slides

- Github repo link https://github.com/kneeshit/STT_Assignments/tree/main/Lab10
- Microsoft. *Inheritance in C# and .NET*. Microsoft Learn. Available at: [https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/inheritance:contentReference\[oaicite:0\]index=0](https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/inheritance:contentReference[oaicite:0]index=0)
- Microsoft. *Constructors (C# programming guide)*. Microsoft Learn. Available at: [https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constructors:contentReference\[oaicite:1\]index=1](https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constructors:contentReference[oaicite:1]index=1)